



Uber Database System

Uber

<i>Trainee Name</i>	<i>Instructor Name</i>	<i>Assistant Name</i>
<i>Maysun Salah</i>	<i>Majd Nasser</i>	<i>Abdulmajed Almutairi</i>

Introduction

Uber is a global ride-sharing platform that connects passengers with nearby drivers through a mobile application. The system manages real-time trip requests, dynamic pricing, driver availability, and secure payment processing. This database design models four core entities of the **Uber platform: Users, Drivers, Trips, and Payments**, linked together through foreign key relationships.

The purpose of this database is to store and manage data accurately while maintaining relationships between tables. It also helps reduce data redundancy, improve data consistency, and support the main operations of the Uber system such as booking rides, assigning drivers, and tracking payments.

Tables Overview

Table	Main Fields	Description
Users	id, full_name, email, gender, country_code	Stores registered passenger accounts
drivers	id, full_name, license, rating, is_available	Stores driver profiles and availability
trips	id, user_id, driver_id, distance, status, created_at	Records each ride and links user to driver
payments	id, trip_id, amount, method, status	One payment record per completed trip

Database Schema Diagram

This section shows the database schema of the Uber system. The first diagram is a simplified design created before writing the queries, while the second one is generated using DataGrip.

Both diagrams help illustrate the structure of the system and the relationships between the main tables: Users, Drivers, Trips, and Payments.

1. Initial Database Schema Design :

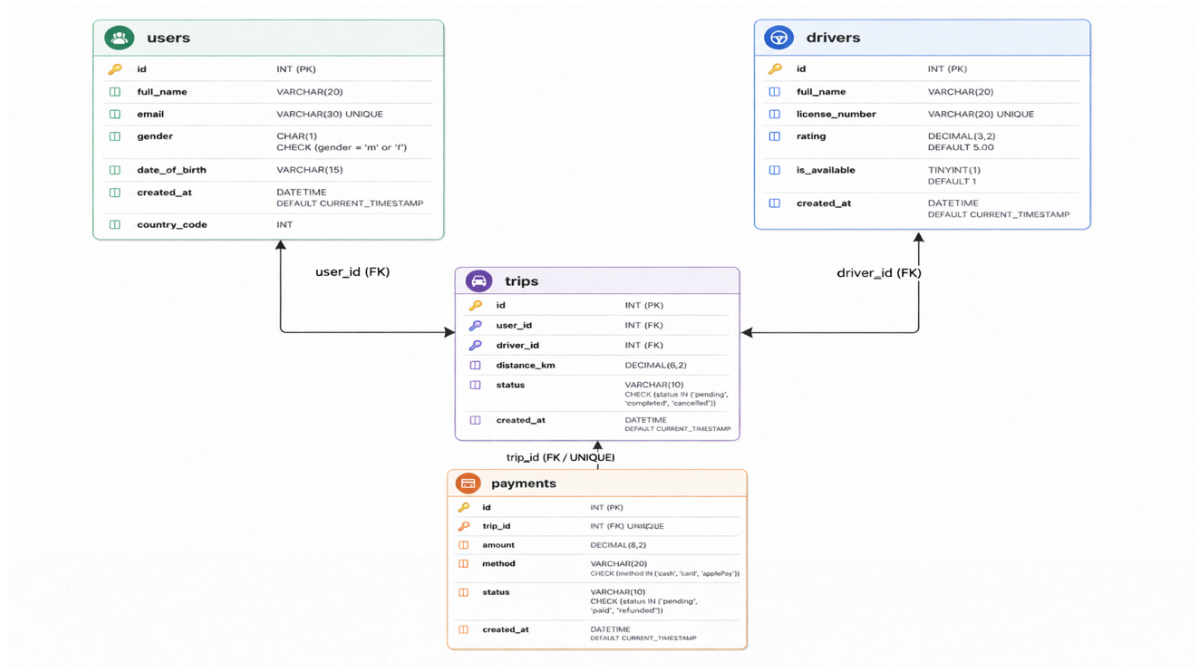


Figure 1 — Initial Uber Database Schema Digeam

2. DataGrip Database Schema :

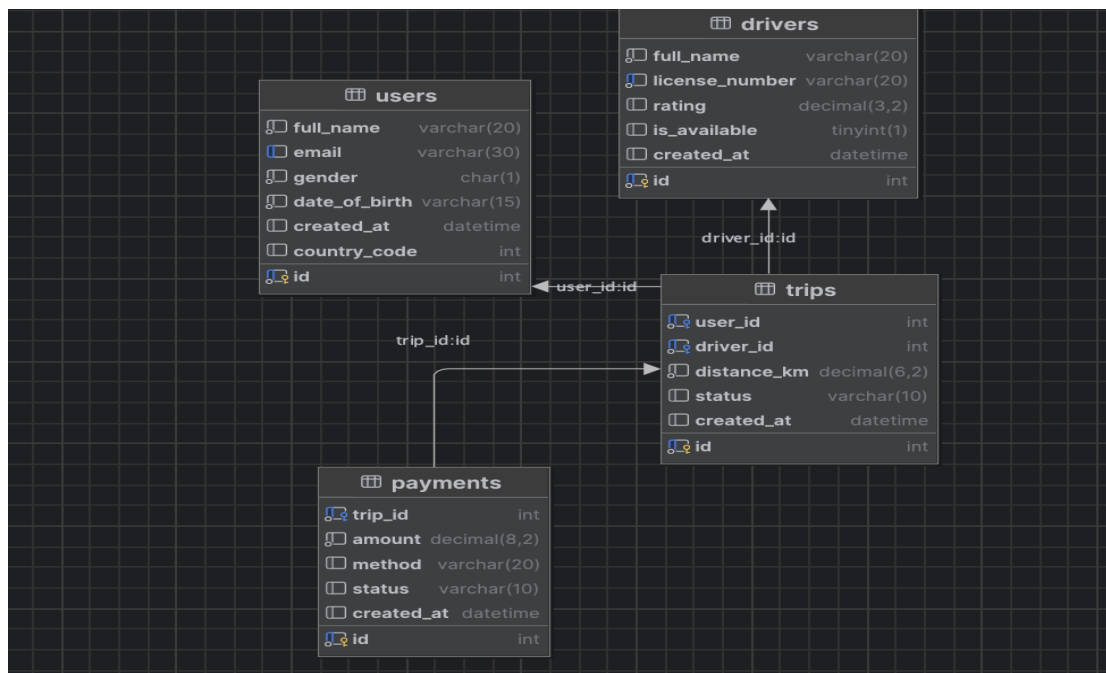


Figure 2 — Uber Database Schema Using DataGrip

Queries

In this project, SQL queries were used to manage the database.

- **DDL** was used to create the database structure (tables, keys, constraints).
- **DML** was used to insert, update, and delete data.
- **CRUD** operations were applied: Create, Read, Update, Delete.

These queries show how the Uber database handles and manages data efficiently

```
-- create the database (DDL)

create database uber;

-- select the database to use it

use uber;

-- users table (stores passenger info)

create table users (
  id int primary key,
  full_name varchar(20) not null,
  email varchar(30) unique,
  gender char(1) not null check(gender = 'm' or gender = 'f'),
  date_of_birth varchar(15) not null,
  created_at datetime,
  country_code int
);

-- drivers table (stores driver profiles)

create table drivers (
  id int primary key,
  full_name varchar(20) not null,
  license_number varchar(20) unique not null,
  rating decimal(3,2) default 5.00,
  is_available boolean default true,
  created_at datetime
);

-- trips table (each ride links a user and a driver)
```

```

create table trips (
  id int primary key,
  user_id int not null,
  driver_id int not null,
  distance_km decimal(6,2) not null,
  status varchar(10) check(status = 'pending' or status = 'completed' or status
= 'cancelled'),
  created_at datetime,

  -- link trip to user and driver

foreign key (user_id) references users(id),
foreign key (driver_id) references drivers(id)
);

-- payments table (one payment per trip)

create table payments (
  id int primary key,
  trip_id int not null unique,
  amount decimal(8,2) not null,
  method varchar(20) check(method = 'cash' or method = 'card' or method =
'applePay'),
  status varchar(10) check(status = 'pending' or status = 'paid' or status =
'refunded'),
  created_at datetime,

  -- link payment to trip

foreign key (trip_id) references trips(id));

-- add default timestamps after table creation

alter table users modify created_at datetime default current_timestamp;
alter table drivers modify created_at datetime default current_timestamp;
alter table trips modify created_at datetime default current_timestamp;
alter table payments modify created_at datetime default current_timestamp;

-- insert users data (DML CRUD)

insert into users (id, full_name, email, gender, date_of_birth) values
(1, 'maysun alharbi', 'maysun@gmail.com', 'f', '2001-07-09'),
(2, 'muath salah', 'muath@gmail.com', 'm', '2002-05-05'),
(3, 'luluah', 'lulu@gmail.com', 'f', '2006-08-20'),

```

```
(4, 'saleh', 'saleh@gmail.com', 'm', '2003-01-15'),  
(5, 'laten', 'laten@gmail.com', 'f', '2002-09-30');
```

-- insert drivers data

```
insert into drivers (id, full_name, license_number, rating, is_available) values  
(1, 'faisal ', 'SA-001', 4.90, true),  
(2, 'waeel', 'SA-002', 4.75, true),  
(3, 'yousef', 'SA-003', 4.60, false),  
(4, 'omar', 'SA-004', 4.85, true),  
(5, 'adel', 'SA-005', 4.50, false);
```

-- insert trips data

```
insert into trips (id, user_id, driver_id, distance_km, status) values  
(1, 1, 1, 12.5, 'completed'),  
(2, 2, 2, 7.3, 'completed'),  
(3, 3, 1, 20.0, 'completed'),  
(4, 4, 3, 5.0, 'cancelled'),  
(5, 5, 4, 15.8, 'completed');
```

-- insert payments data

```
insert into payments (id, trip_id, amount, method, status) values  
(1, 1, 45.00, 'card', 'paid'),  
(2, 2, 27.50, 'applePay', 'paid'),  
(3, 3, 72.00, 'cash', 'paid'),  
(4, 4, 0.00, 'card', 'refunded'),  
(5, 5, 56.80, 'applePay', 'paid');
```

-- update users

```
update users set email = 'mays@gmail.com' where id = 1;  
update users set full_name = 'moaad salah' where id = 2;  
select *from users;
```

-- update drivers

```
update drivers set rating = 4.95 where id = 1;  
select *from drivers;
```

-- update payments

```
update payments set status = 'paid' where id = 4;  
select *from payments;
```

```
-- delete some data
```

```
delete from payments where trip_id = 4;
```

```
--update users on country_code filed
```

```
update users set country_code = 966 where id = 1;  
update users set country_code = 966 where id = 2;  
update users set country_code = 977 where id = 3;  
update users set country_code = 959 where id = 4;  
update users set country_code = 974 where id = 5;
```

```
select *from users;
```